

DAY 1

API Design + Integrations

Printable refresh + one full design practice

Goal for today: Refresh practical API design fundamentals, then answer one integration design question out loud using a senior-level structure.

Suggested 60-minute use

- 35-40 min: read/refresh the API design material.
- 15-20 min: answer the design question out loud without looking at the answer.
- 5 min: review the answer checklist and repeat only the weak parts.

Interview mental checklist

Contract -> Auth -> Validation -> Sync/Async -> Timeout -> Retry
-> Idempotency -> Rate limiting -> Failure isolation
-> Persistence -> Reconciliation -> Observability

Rule: Do not jump straight to Kafka, Kubernetes, or a specific technology. Clarify requirements and failure semantics first.

1. REST resources and HTTP semantics

Think in resources, not action names.

GOOD
 GET /reservations/123
 POST /reservations
 PATCH /reservations/123
 DELETE /reservations/123

LESS REST-LIKE
 POST /createReservation
 POST /updateReservation

Nested resources are fine when the relationship matters:

GET /properties/42/reservations
 GET /properties/42/rooms

Verb	Typical use	Safe?	Idempotent?
GET	Read	Yes	Yes
POST	Create / command	No	Usually no
PUT	Replace full resource	No	Yes
PATCH	Partial modification	No	Not guaranteed
DELETE	Remove	No	Yes, semantically

Know this distinction: Safe means the request should not change state. Idempotent means repeating the same request has the same final effect.

2. Status codes you should know

Code	When to use it
200 OK	Successful read/update with body
201 Created	Resource created
202 Accepted	Accepted for async processing
204 No Content	Success, nothing to return
304 Not Modified	Cache validation
400 Bad Request	Malformed request / invalid syntax
401 Unauthorized	Missing or invalid authentication
403 Forbidden	Authenticated, but not allowed
404 Not Found	Resource does not exist
409 Conflict	Operation conflicts with current resource state
412 Precondition Failed	ETag/version precondition failed
422 Unprocessable Content	Structurally valid, but domain validation failed

Code	When to use it
429 Too Many Requests	Rate limited
500 Internal Server Error	Unexpected server failure
502 Bad Gateway	Bad/failing upstream response
503 Service Unavailable	Temporarily unavailable
504 Gateway Timeout	Upstream timed out

Fast examples: Wrong JSON -> 400 | no token -> 401 | valid token, wrong permission -> 403 | async job accepted -> 202 | partner rate limit -> 429.

3. Pagination, filtering, sorting

Do not casually return hundreds of thousands of rows.

Offset pagination

```
GET /reservations?limit=100&offset=200
```

- Easy to implement and understand.
- Large offsets can become expensive.
- Concurrent inserts/deletes can cause duplicates or skipped rows.
- Fine for small admin screens or stable datasets.

Cursor / keyset pagination

```
GET /reservations?limit=100&cursor=eyJpZCI6...
```

```
{
  "items": [...],
  "next_cursor": "xyz"
}
```

- Better for large or frequently changing datasets.
- Uses a stable sort key, for example created_at + id.
- Cursor should be opaque to clients.

```
SELECT ...
WHERE (created_at, id) < (:last_created_at, :last_id)
ORDER BY created_at DESC, id DESC
LIMIT 100;
```

Interview default: For a large external API with changing data, cursor pagination is usually the more defensible default.

Filtering/sorting example:

```
GET /reservations?property_id=42&status=confirmed&created_after=...&sort=-created_at&limit=100
```

Important: Do not allow arbitrary expensive sorting/filtering on unindexed database fields just because the API can expose it.

4. Versioning and backward compatibility

```
/api/v1/reservations
```

- Adding an optional field is usually backward compatible.
- Changing field type/shape or removing a field is breaking.
- For partner APIs, deprecation needs documentation, migration time, and usage monitoring.

SAFE-ish ADDITION

```
{ "id": "123", "status": "confirmed", "new_field": "..." }
```

BREAKING CHANGE

```
"price": "100" -> "price": { "value": 100, "currency": "EUR" }
```

5. Authentication and authorization

Authentication = who are you? Authorization = what are you allowed to do?

API keys

- Simple for machine-to-machine access.
- Need secure storage, rotation, revocation, and scoped permissions.

Authorization: Bearer <api-key>

OAuth2

- Good when a third-party app needs delegated access on behalf of a customer.
- Typical flow: authorize -> code -> access token -> API calls -> refresh token.

JWT

- JWT is a token format, not a replacement for OAuth.
- Validate signature, issuer, audience, expiry, and claims/scopes.

```
{
  "sub": "partner-123",
  "scope": ["reservations:read"],
  "exp": 1234567890
}
```

6. Error design

Keep errors machine-readable and stable.

```
{
  "error": {
    "code": "RESERVATION_ALREADY_CANCELLED",
    "message": "Reservation is already cancelled",
    "request_id": "req_abc123"
  }
}
```

- Clients should depend on stable error codes, not parse human text.
- Include a request/correlation ID for support and debugging.

7. Optimistic concurrency

Protect against lost updates when two clients modify the same resource.

Response:

ETag: "version-15"

Update request:

If-Match: "version-15"

Already changed? -> 412 Precondition Failed

UPDATE reservation

```
SET ..., version = version + 1
WHERE id = 123 AND version = 15;
```

8. Timeouts, retries, backoff, jitter

Timeouts

- Never call a third party with an unlimited timeout.
- Think about connect timeout, read timeout, and overall request deadline.
- A timeout does NOT prove the remote operation failed.

Classic ambiguity: You send "create reservation". The partner commits it, but your connection dies before the response arrives. Retrying blindly can duplicate the operation.

Retry what is likely transient

- Network errors, timeouts, 429, 502, 503, often 504.
- Usually do not retry 400/401/403/404/validation failures without a domain-specific reason.
- Retries must be bounded.

Exponential backoff + jitter

1s -> 2s -> 4s -> 8s -> 16s

with jitter: 1.2s -> 1.7s -> 4.6s -> 7.1s ...

Why jitter? Without jitter, thousands of workers can retry at the same instant and cause a second outage.

9. Rate limiting

HTTP/1.1 429 Too Many Requests
Retry-After: 30

1. Respect Retry-After.
2. Apply exponential backoff and jitter.
3. Control local concurrency so recovery does not flood the partner.
4. Monitor rate-limit responses and backlog.

10. Circuit breaker

```
CLOSED -> normal calls
| many failures
v
OPEN -> fail fast; do not call dependency
| cooldown
v
HALF-OPEN -> allow a few probes
| success -> CLOSED
| failure -> OPEN
```

Purpose: Protect your own service from exhausting threads, connections, and latency budget when a dependency is unhealthy.

11. Idempotency and effective-once behavior

A POST such as payment/reservation creation may execute twice if retried. Use an idempotency key when the operation must not duplicate.

```
POST /payments
Idempotency-Key: 7e26...
```

server stores: key -> operation/result
 repeated request -> original result, not a second charge

Interview wording: Prefer: "I design for at-least-once delivery and make operations idempotent, producing effectively-once business behavior." Avoid casually claiming true end-to-end exactly-once.

Typical mechanisms:

- Unique constraint on business key or event_id.
- Processed-event table / dedup store.
- Upsert / ON CONFLICT behavior.
- Version checks so stale events cannot overwrite newer state.

12. Reconciliation

Retries are not enough. Some failures remain ambiguous or exceed retry windows.

- Periodically compare local state with partner state.
- Fetch missing/changed records and repair drift.
- Use reconciliation after outages or when delivery guarantees are imperfect.

Senior point: A resilient integration normally has both a fast path (events/webhooks) and a repair path (reconciliation).

13. Webhooks

Webhooks reduce polling: the producer pushes changes to the partner.

```
{
  "event_id": "evt_123",
  "event_type": "reservation.updated",
  "created_at": "...",
  "data": {...}
}
```

Design for reality:

- Duplicates.
- Delayed delivery.
- Out-of-order delivery.
- Receiving endpoint unavailable.
- Retry storms.

Good webhook design:

- Stable event_id for deduplication.
- Signature/HMAC verification.
- Return quickly; process asynchronously.
- Retry transient failures with backoff.
- Provide an API/reconciliation path to repair missed events.

14. Sync vs async

Use synchronous calls when the caller needs an immediate answer and the work is bounded.

GET reservation
 validate a request
 small lookup

Use async processing when work is slow, bursty, or failure-prone, and completion does not need to block the caller.

POST /exports
 -> 202 Accepted
 { "job_id": "123", "status": "pending" }

GET /exports/123

Useful pattern: Hybrid systems can return fresh data synchronously while asynchronously publishing/persisting the same result for future reads.

15. REST vs GraphQL

	Good fit	Trade-offs
REST	Simple resource APIs; partner integrations; clear HTTP semantics; easier caching	May require multiple calls; fixed response shapes
GraphQL	Clients need flexible nested data and selective fields	Query complexity, auth complexity, N+1 risk, caching is harder, expensive queries must be controlled

Do not say: "GraphQL is better than REST." Say what requirements make one a better fit.

16. Practice design question

Do this without looking at the answer first. Speak for 5-8 minutes. Start with clarifying questions, then design the happy path and failure path.

Scenario

Cloudbeds needs to synchronize reservations with an external booking partner. When a reservation is created, updated, or cancelled in Cloudbeds, the partner must receive the change. The partner API sometimes goes offline, is limited to 100 requests/second, can take 5-10 seconds to respond, and may receive duplicate requests. We must avoid losing updates and minimize inconsistent reservation state.

First: clarify requirements

- Does Cloudbeds need partner confirmation before the booking is accepted?
- Does the partner support idempotency keys?
- Does the partner require strict ordering per reservation?
- Can events be eventually consistent, and what is the tolerated delay?
- Is there a partner API to query current state for reconciliation?

Whiteboard sketch

```

Cloudbeds API
|
| DB transaction
v
Reservation DB + Outbox
|
v
Message broker
|
v
Integration workers
| rate limit / timeout / retry / breaker
v
Partner API

Failure after retries -> retry topic / DLQ / manual review
Periodic reconciliation -> compare Cloudbeds vs partner
  
```

17. Strong answer walkthrough

1) Keep the primary booking transaction independent where possible

Do not keep the user request open for 5-10 seconds waiting on an unreliable partner unless business semantics require that. Persist the reservation change and the intent to publish in one local transaction.

2) Use a transactional outbox

```

BEGIN
  UPDATE reservation ... version = 8
  INSERT outbox(event_id, reservation_id, version, type, payload)
COMMIT
  
```

A background publisher forwards outbox events to the broker. This avoids the dual-write problem: DB commit succeeds but message publish silently fails, or vice versa.

3) Partition/order by reservation when needed

- Use reservation_id as a partitioning/ordering key if updates for one reservation must stay ordered.
- Do not require global ordering; it destroys throughput unnecessarily.

4) Make delivery idempotent

- Every event has an immutable event_id and reservation version.
- If the partner supports idempotency keys, send event_id as the key.
- If it does not, maintain local delivery state and reconcile ambiguous outcomes.

```

{
  "event_id": "evt-123",
  
```

```

"reservation_id": "res-42",
"version": 8,
"type": "reservation.updated"
}

```

5) Control rate and concurrency

- Token bucket / rate limiter at ≤ 100 req/s, ideally leave headroom.
- Cap concurrent in-flight requests because 5-10 second latency can otherwise create thousands of open connections.
- Track queue depth so you can see backlog forming.

6) Retry only transient failures

- Timeout/network/429/5xx -> bounded exponential backoff + jitter.
- 4xx validation/auth failures -> do not blindly retry; route to investigation or DLQ.
- Respect Retry-After on 429.

7) Circuit breaker

If the partner is failing consistently, open the breaker and let messages remain queued rather than consuming all workers waiting on timeouts.

8) Handle poison messages

After bounded retries, move permanently failing events to a DLQ/review path with enough metadata to replay after correction.

9) Reconcile

Run periodic comparison against the partner because even a well-designed distributed integration can have ambiguous failures, manual changes, or partner-side bugs. Repair missing or stale reservations.

10) Observability

- Success/failure rate per partner.
- p50/p95/p99 partner latency.
- 429 and 5xx rate.
- Queue depth and oldest-message age.
- Retry count and DLQ size.
- Reconciliation drift count.
- Correlation IDs / event IDs across logs and traces.

Senior summary sentence: "I would isolate the unreliable dependency behind an asynchronous delivery pipeline, make each update replay-safe, bound retries and concurrency, and have reconciliation as the correctness repair mechanism."

18. Follow-up questions they may ask on this design

Question	Short answer
Why not just call the partner synchronously?	Because its 5-10s latency and outages would directly consume our latency budget and availability. Async decouples the systems unless the business requires immediate partner confirmation.
What if publishing to Kafka fails after the DB update?	Transactional outbox: the event is persisted in the same DB transaction and retried by an outbox publisher.
What if the partner processed the request but our call timed out?	Treat the result as ambiguous. Retry only with an idempotency key if supported; otherwise query partner state or let reconciliation resolve it.
How do you preserve order?	Partition/order by reservation_id and include a monotonic version. Consumers reject or ignore stale versions.
How do you avoid overwhelming the partner after a 3-hour outage?	Rate-limit recovery, cap concurrency, drain backlog gradually, use jitter, and monitor oldest-message age.
What if one event is permanently invalid?	Bound retries, send it to DLQ/manual review, and do not let it block unrelated messages.
Exactly once?	I would not depend on true end-to-end exactly once. I would use at-least-once delivery plus idempotent operations, versioning, and reconciliation.
What if the partner has no webhook/idempotency/reconciliation API?	The guarantee is weaker. I would explicitly surface that limitation, persist delivery attempts, avoid unsafe retries where outcome is ambiguous, and define an operational/manual repair path.

19. 5-minute final recall sheet

- REST: resources, correct verbs, stable errors, backward compatibility.
- Pagination: cursor/keyset for large changing datasets.
- Auth: API key vs OAuth2; authn != authz; JWT is a token format.
- Resiliency: deadlines, bounded retries, backoff + jitter, circuit breaker.
- Correctness: idempotency + versions + reconciliation.
- Webhooks: duplicates, delays, ordering, signatures, async handling.
- Async design: transactional outbox -> broker -> workers -> partner.
- Observability: latency, error rate, queue depth/age, retries, DLQ, drift.

Do not over-answer. Start with the architecture and failure semantics. Give implementation detail only when they drill down.